OrarioClasse — Terzo passo: interfaccia grafica WPF

1. Obiettivo

Nei primi due passi abbiamo realizzato il **modello dati** (`Tessera`, `Posizione`, `Orario`) e il **motore degli scambi** (`ValidatoreOrario`, `MotoreScambi`). In questo passo costruiamo una finestra Windows con **WPF e XAML** che visualizza l'orario e permette di selezionare due celle e chiedere uno scambio.

La classe ha 5 giorni, 8 ore al giorno, 34 tessere e 6 celle vuote. Ogni tessera contiene **materia** e **aula**. Le lezioni di un giorno devono essere consecutive a partire dalla prima ora: non possono esserci buchi. Le celle vuote sono selezionabili.

Regola architetturale: la finestra raccoglie i clic e aggiorna la visualizzazione, ma **non decide** se uno scambio è valido. Per questo chiama `MotoreScambi`.

2. Preparazione del progetto

In Visual Studio creare un progetto **App WPF** denominato `OrarioClasse`, con C# e .NET 8 (o una versione successiva di .NET compatibile con WPF). WPF funziona su Windows. Nel file `OrarioClasse.csproj` mantenere le impostazioni WPF e, per coerenza con i primi passi, disabilitare l'analisi dei riferimenti nullable:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>
</Project>
```

Struttura dei file:

```
OrarioClasse/
├── Model/
│   ├── Tessera.cs
│   ├── Posizione.cs
│   └── Orario.cs
├── Engine/
│   ├── ValidatoreOrario.cs
│   └── MotoreScambi.cs
├── MainWindow.xaml
├── MainWindow.xaml.cs
├── App.xaml
└── App.xaml.cs
```

Nota: `MainWindow.xaml` e `MainWindow.xaml.cs` restano nella cartella principale del progetto, come nel modello predefinito di Visual Studio. Le cartelle `Model` ed `Engine` contengono il codice dei primi due passi **senza modifiche**. Per semplicità, in questo passo non introduciamo ancora una cartella `View` né un ViewModel.

3. Attività A — Costruire la finestra

3.1 XAML: descrivere la finestra

Sostituire il contenuto di `MainWindow.xaml` con:

```
<Window x:Class="OrarioClasse.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Orario di classe" Height="650" Width="900"
        MinHeight="500" MinWidth="650">
    <DockPanel Margin="15">
        <TextBlock DockPanel.Dock="Top"
                   Text="Orario della classe — 5 giorni, 8 ore"
                   FontSize="20" FontWeight="Bold"
                   Margin="0,0,0,12" />

        <StackPanel DockPanel.Dock="Bottom"
                    Orientation="Horizontal"
                    Margin="0,12,0,0">
            <Button x:Name="btnScambia"
                    Content="Scambia"
                    Width="110" Height="35"
                    IsEnabled="False"
                    Click="Scambia_Click" />
            <TextBlock x:Name="txtStato"
                       Text="Seleziona due celle."
                       VerticalAlignment="Center"
                       Margin="15,0,0,0" />
        </StackPanel>

        <Grid x:Name="grigliaOrario" />
    </DockPanel>
</Window>
```

Il `DockPanel` dispone il titolo in alto, il pulsante e il messaggio in basso, e lascia alla `Grid` lo spazio centrale. La `Grid` inizialmente è vuota: i pulsanti saranno generati da C#.

3.2 Generazione dinamica dei 40 pulsanti

Per mostrare anche le intestazioni, la griglia grafica avrà **6 colonne e 9 righe**: una colonna per i numeri delle ore e una riga per i nomi dei giorni. I pulsanti dell'orario restano esattamente **5 × 8 = 40**.

La finestra memorizzerà i pulsanti in una matrice `Button[5,8]`. Questa matrice appartiene alla GUI; la matrice `Tessera[5,8]` rimane nel modello.

Il metodo `CreaScacchiera()` nel codice completo della sezione 6:

1. Aggiunge righe e colonne alla `Grid`.
2. Crea le intestazioni dei giorni e delle ore.
3. Crea 40 pulsanti con due righe di testo (materia e aula).
4. Associa a ogni pulsante la sua `Posizione` attraverso la proprietà `Tag`.
5. Collega ogni pulsante allo stesso gestore `Cella_Click`.

`Grid.SetRow()` e `Grid.SetColumn()` assegnano la posizione del controllo nella griglia grafica. Gli indici visuali sono aumentati di 1 perché la prima riga e la prima colonna sono riservate alle intestazioni.

3.3 Approfondimento — Come funziona la `Grid` in `CreaScacchiera()`

La `Grid` di WPF è un **contenitore grafico**: permette di disporre controlli (pulsanti, testi e altri elementi) in righe e colonne. Non è la matrice che conserva le tessere. Nel progetto utilizziamo infatti tre strutture diverse:

Struttura	Tipo	Che cosa contiene
`celle` della classe `Orario`	`Tessera[5,8]`	I dati delle lezioni, oppure `null`
`pulsanti` della finestra	`Button[5,8]`	I riferimenti ai 40 pulsanti
`grigliaOrario` della finestra	`Grid` WPF	I controlli grafici e la loro disposizione

La `Grid` non è una matrice C#: è un oggetto che gestisce il layout dei controlli. Il suo nome `grigliaOrario` deriva dall'attributo `x:Name` scritto nel file XAML:

```
<Grid x:Name="grigliaOrario" />
```

Grazie a `x:Name`, possiamo riferirci a questo oggetto dal file `MainWindow.xaml.cs`.

A. Definire le colonne

```
for (int colonna = 0; colonna < 6; colonna++)
    grigliaOrario.ColumnDefinitions.Add(new ColumnDefinition());
```

Il ciclo viene eseguito sei volte. Ogni volta `new ColumnDefinition()` crea un oggetto che descrive una colonna, mentre `ColumnDefinitions.Add(...)` lo aggiunge alla raccolta delle colonne della `Grid`.

Servono **sei colonne**, non cinque, perché la prima (`0`) contiene le etichette delle ore e le successive (`1`–`5`) rappresentano i giorni da lunedì a venerdì. Non assegniamo una larghezza esplicita: con le impostazioni predefinite, le colonne condividono lo spazio disponibile.

B. Definire le righe

```
for (int riga = 0; riga < 9; riga++)
```

```
    grigliaOrario.RowDefinitions.Add(new RowDefinition());
```

Il funzionamento è analogo: `new RowDefinition()` crea la definizione di una riga e `RowDefinitions.Add(...)` la aggiunge alla `Grid`.

Servono **nove righe**: la prima (`0`) contiene i nomi dei giorni e le altre (`1`–`8`) corrispondono alle otto ore. Anche le altezze sono gestite automaticamente dalla `Grid`.

Lo schema delle coordinate grafiche è il seguente:

```
             Colonna della Grid
           0     1     2     3     4     5
         +-----+-----+-----+-----+-----+-----+
Riga  0  |     | Lun | Mar | Mer | Gio | Ven |
      1  | 1ª  |  B  |  B  |  B  |  B  |  B  |
      2  | 2ª  |  B  |  B  |  B  |  B  |  B  |
      3  | 3ª  |  B  |  B  |  B  |  B  |  B  |
      4  | 4ª  |  B  |  B  |  B  |  B  |  B  |
      5  | 5ª  |  B  |  B  |  B  |  B  |  B  |
      6  | 6ª  |  B  |  B  |  B  |  B  |  B  |
      7  | 7ª  |  B  |  B  |  B  |  B  |  B  |
      8  | 8ª  |  B  |  B  |  B  |  B  |  B  |
         +-----+-----+-----+-----+-----+-----+

B = Button (una cella dell'orario)
```

C. Posizionare le intestazioni dei giorni

Il primo ciclo crea cinque `TextBlock`, uno per ciascun giorno:

```
for (int giorno = 0; giorno < 5; giorno++)
{
    TextBlock intestazione = new TextBlock();
    intestazione.Text = giorni[giorno];
    intestazione.FontWeight = FontWeights.Bold;
    intestazione.HorizontalAlignment = HorizontalAlignment.Center;
    intestazione.VerticalAlignment = VerticalAlignment.Center;

    Grid.SetRow(intestazione, 0);
    Grid.SetColumn(intestazione, giorno + 1);
    grigliaOrario.Children.Add(intestazione);
}
```

- `new TextBlock()` crea un controllo che mostra testo.
- `Text` imposta la scritta (`Lun`, `Mar` e così via).
- `FontWeight` imposta il grassetto e le due proprietà `Alignment` centrano il testo nella cella grafica.
- `Grid.SetRow(intestazione, 0)` indica che il controllo deve essere nella **riga 0**.

- `Grid.SetColumn(intestazione, giorno + 1)` sceglie la colonna: lunedì va in colonna 1, martedì in colonna 2 e così via.
- `grigliaOrario.Children.Add(intestazione)` **inserisce realmente** il controllo nella `Grid`.

Distinzione importante: `Grid.SetRow()` e `Grid.SetColumn()` indicano *dove* collocare un controllo; `Children.Add()` indica *quale controllo* appartiene alla griglia. Sono operazioni diverse e servono entrambe.

D. Posizionare i numeri delle ore

Il secondo ciclo crea otto `TextBlock`:

```
for (int ora = 0; ora < 8; ora++)
{
    TextBlock numeroOra = new TextBlock();
    numeroOra.Text = (ora + 1) + "ª";
    numeroOra.HorizontalAlignment = HorizontalAlignment.Center;
    numeroOra.VerticalAlignment = VerticalAlignment.Center;

    Grid.SetRow(numeroOra, ora + 1);
    Grid.SetColumn(numeroOra, 0);
    grigliaOrario.Children.Add(numeroOra);
}
```

Questi controlli sono tutti nella **colonna 0**. La loro riga è `ora + 1`: quando `ora` vale `0`, il testo `1ª` viene inserito nella riga 1; quando `ora` vale `7`, il testo `8ª` va nella riga 8. L'espressione `ora + 1` serve sia a mostrare numeri di ora comprensibili all'utente sia a saltare la riga delle intestazioni.

E. Creare e collocare i 40 pulsanti

Il doppio ciclo attraversa le cinque giornate e le otto ore:

```
for (int giorno = 0; giorno < 5; giorno++)
{
    for (int ora = 0; ora < 8; ora++)
    {
        Button pulsante = new Button();
        pulsante.Margin = new Thickness(3);
        pulsante.Tag = new Posizione(giorno, ora);
        pulsante.Click += Cella_Click;

        Grid.SetRow(pulsante, ora + 1);
        Grid.SetColumn(pulsante, giorno + 1);
        grigliaOrario.Children.Add(pulsante);

        pulsanti[giorno, ora] = pulsante;
    }
}
```

A ogni iterazione:

1. `new Button()` crea un nuovo pulsante.
2. `Margin = new Thickness(3)` lascia uno spazio di 3 unità WPF intorno al pulsante, separandolo visivamente dagli altri.
3. `Tag = new Posizione(giorno, ora)` associa al pulsante le **coordinate del modello**. `Tag` può contenere un oggetto; qui contiene un oggetto `Posizione`.
4. `Click += Cella_Click` collega il clic del pulsante al metodo che gestirà la selezione. Tutti i 40 pulsanti utilizzano lo stesso metodo.
5. `Grid.SetRow(...)` e `Grid.SetColumn(...)` assegnano le coordinate **grafiche**, aumentate di uno per lasciare spazio alle intestazioni.
6. `Children.Add(pulsante)` aggiunge il pulsante alla griglia visibile.
7. `pulsanti[giorno, ora] = pulsante` conserva il riferimento al pulsante nella matrice della finestra, per ritrovarlo quando occorre aggiornare il testo o il colore.

Per esempio, per `giorno = 2` e `ora = 3` (mercoledì, quarta ora):

Elemento	Coordinate
Posizione nel modello	`(2, 3)`
Elemento della matrice `pulsanti`	`pulsanti[2, 3]`
Posizione nella `Grid` WPF	riga `4`, colonna `3`

La differenza tra le coordinate è intenzionale: **nel modello non ci sono intestazioni; nella `Grid` sì**.

F. Perché non si imposta subito `Content`?

`CreaScacchiera()` ha una sola responsabilità: **creare e disporre i controlli grafici**. Il testo dei pulsanti viene assegnato successivamente da `AggiornaScacchiera()`, che legge le tessere dall'oggetto `Orario`.

Questa separazione evita di ricreare 40 pulsanti dopo ogni scambio:

- `CreaScacchiera()` viene chiamato **una volta** all'avvio;
- `AggiornaScacchiera()` può essere chiamato **più volte** per aggiornare materia e aula;
- `AggiornaSelezione()` può cambiare il colore dei pulsanti senza modificarne la posizione.

Da ricordare: la `Grid` organizza i controlli; la matrice `pulsanti` permette di recuperarli; l'oggetto `Orario` contiene i dati effettivi. Il motore lavora sui dati, non sui pulsanti.

4. Attività B — Selezionare due celle

Per la selezione utilizziamo due variabili di tipo `Posizione`, inizialmente `null`:

```
private Posizione primaSelezione;
private Posizione secondaSelezione;
```

Quando l'utente fa clic su una cella:

- se non ci sono selezioni, diventa la prima;
- se è già selezionata, il clic la deseleziona;
- se c'è una sola selezione, diventa la seconda;
- se sono già selezionate due celle, un nuovo clic avvia una nuova selezione.

Le celle selezionate sono evidenziate in azzurro. Il pulsante **Scambia** si abilita soltanto quando sono selezionate due celle distinte. La posizione di una cella si recupera dal `Tag` del pulsante cliccato. Per questo esempio il `Tag` contiene direttamente un oggetto `Posizione`.

Evento: `Cella_Click(object sender, RoutedEventArgs e)` riceve come `sender` il pulsante che ha generato il clic. Il cast `(Button)sender` consente di accedere al suo `Tag`.

5. Attività C — Collegare la finestra al motore

Quando l'utente preme **Scambia**, la finestra chiama:

```
bool riuscito = motore.Scambia(
    orario, primaSelezione, secondaSelezione);
```

Se `riuscito` è `true`, il motore ha lasciato applicato lo scambio; se è `false`, il motore ha ripristinato le tessere originali. La finestra mostra l'esito, azzera la selezione e aggiorna tutti i pulsanti leggendo il modello.

Importante: non bisogna scambiare direttamente il contenuto dei pulsanti. È il modello che conserva la situazione reale; la GUI ne mostra una rappresentazione.

6. Codice completo — MainWindow.xaml.cs

Il codice seguente realizza tutte e tre le attività. Utilizza solo classi, costruttori, array, cicli, `if`, eventi e metodi tradizionali. Si appoggia alle cinque classi già costruite nei passi precedenti.

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using OrarioClasse.Model;
using OrarioClasse.Engine;

namespace OrarioClasse
{
    public partial class MainWindow : Window
    {
        private Orario orario;
        private MotoreScambi motore;
        private Button[,] pulsanti;

        private Posizione primaSelezione;
        private Posizione secondaSelezione;

        public MainWindow()
        {
```

```
            InitializeComponent();

            orario = new Orario();
            motore = new MotoreScambi();
            pulsanti = new Button[5, 8];

            CaricaOrarioEsempio();
            CreaScacchiera();
            AggiornaScacchiera();
            AggiornaSelezione();
        }

        // Crea 34 tessere senza buchi: 7+7+7+7+6.
        private void CaricaOrarioEsempio()
        {
            string[] materie = {
                "Matematica", "Informatica", "Italiano",
                "Inglese", "Sistemi", "TPS", "Storia"
            };

            for (int giorno = 0; giorno < 5; giorno++)
            {
                int lezioni = 7;
                if (giorno == 4)
                    lezioni = 6;

                for (int ora = 0; ora < lezioni; ora++)
                {
                    string materia = materie[(giorno + ora) % materie.Length];
                    string aula = "A" + (ora + 1);
                    Tessera tessera = new Tessera(materia, aula);
                    orario.Scrivi(new Posizione(giorno, ora), tessera);
                }
            }
        }

        private void CreaScacchiera()
        {
            string[] giorni = { "Lun", "Mar", "Mer", "Gio", "Ven" };

            // Una colonna per le ore, cinque per i giorni.
            for (int colonna = 0; colonna < 6; colonna++)
                grigliaOrario.ColumnDefinitions.Add(new ColumnDefinition());

            // Una riga per le intestazioni, otto per le ore.
            for (int riga = 0; riga < 9; riga++)
                grigliaOrario.RowDefinitions.Add(new RowDefinition());

            for (int giorno = 0; giorno < 5; giorno++)
            {
                TextBlock intestazione = new TextBlock();
                intestazione.Text = giorni[giorno];
                intestazione.FontWeight = FontWeights.Bold;
                intestazione.HorizontalAlignment = HorizontalAlignment.Center;
```

```
            intestazione.VerticalAlignment = VerticalAlignment.Center;
            Grid.SetRow(intestazione, 0);
            Grid.SetColumn(intestazione, giorno + 1);
            grigliaOrario.Children.Add(intestazione);
        }

        for (int ora = 0; ora < 8; ora++)
        {
            TextBlock numeroOra = new TextBlock();
            numeroOra.Text = (ora + 1) + "ª";
            numeroOra.HorizontalAlignment = HorizontalAlignment.Center;
            numeroOra.VerticalAlignment = VerticalAlignment.Center;
            Grid.SetRow(numeroOra, ora + 1);
            Grid.SetColumn(numeroOra, 0);
            grigliaOrario.Children.Add(numeroOra);
        }

        for (int giorno = 0; giorno < 5; giorno++)
        {
            for (int ora = 0; ora < 8; ora++)
            {
                Button pulsante = new Button();
                pulsante.Margin = new Thickness(3);
                pulsante.Tag = new Posizione(giorno, ora);
                pulsante.Click += Cella_Click;

                Grid.SetRow(pulsante, ora + 1);
                Grid.SetColumn(pulsante, giorno + 1);
                grigliaOrario.Children.Add(pulsante);

                pulsanti[giorno, ora] = pulsante;
            }
        }
    }

    private void AggiornaScacchiera()
    {
        for (int giorno = 0; giorno < 5; giorno++)
        {
            for (int ora = 0; ora < 8; ora++)
            {
                Tessera tessera = orario.Leggi(
                    new Posizione(giorno, ora));

                Button pulsante = pulsanti[giorno, ora];

                if (tessera == null)
                {
                    pulsante.Content = "—";
                }
                else
                {
                    StackPanel contenuto = new StackPanel();
                    TextBlock materia = new TextBlock();
```

```
                    materia.Text = tessera.Materia;
                    materia.FontWeight = FontWeights.Bold;
                    materia.TextAlignment = TextAlignment.Center;

                    TextBlock aula = new TextBlock();
                    aula.Text = tessera.Aula;
                    aula.TextAlignment = TextAlignment.Center;

                    contenuto.Children.Add(materia);
                    contenuto.Children.Add(aula);
                    pulsante.Content = contenuto;
                }
            }
        }
    }

    private bool StessaPosizione(Posizione a, Posizione b)
    {
        if (a == null || b == null)
            return false;

        return a.Giorno == b.Giorno && a.Ora == b.Ora;
    }

    private void Cella_Click(object sender, RoutedEventArgs e)
    {
        Button pulsante = (Button)sender;
        Posizione scelta = (Posizione)pulsante.Tag;

        if (StessaPosizione(scelta, primaSelezione))
        {
            primaSelezione = secondaSelezione;
            secondaSelezione = null;
        }
        else if (StessaPosizione(scelta, secondaSelezione))
        {
            secondaSelezione = null;
        }
        else if (primaSelezione == null || secondaSelezione != null)
        {
            primaSelezione = scelta;
            secondaSelezione = null;
        }
        else
        {
            secondaSelezione = scelta;
        }

        txtStato.Text = "Seleziona due celle e premi Scambia.";
        AggiornaSelezione();
    }

    private void AggiornaSelezione()
    {
```

```
            for (int giorno = 0; giorno < 5; giorno++)
            {
                for (int ora = 0; ora < 8; ora++)
                {
                    Posizione posizione = new Posizione(giorno, ora);
                    Button pulsante = pulsanti[giorno, ora];

                    if (StessaPosizione(posizione, primaSelezione) ||
                        StessaPosizione(posizione, secondaSelezione))
                        pulsante.Background = Brushes.LightSkyBlue;
                    else
                        pulsante.Background = Brushes.WhiteSmoke;
                }
            }

            btnScambia.IsEnabled =
                primaSelezione != null && secondaSelezione != null;
        }

        private void Scambia_Click(object sender, RoutedEventArgs e)
        {
            if (primaSelezione == null || secondaSelezione == null)
                return;

            bool riuscito = motore.Scambia(
                orario, primaSelezione, secondaSelezione);

            if (riuscito)
                txtStato.Text = "Scambio autorizzato.";
            else
                txtStato.Text = "Scambio rifiutato: si creerebbe un buco.";

            primaSelezione = null;
            secondaSelezione = null;
            AggiornaScacchiera();
            AggiornaSelezione();
        }
    }
}
```

Come leggere il codice

- **Campi della finestra:** `orario` e `motore` sono oggetti della logica; `pulsanti` contiene i controlli grafici; `primaSelezione` e `secondaSelezione` conservano le scelte dell'utente.
- **Costruttore:** inizializza la finestra, crea i dati di esempio, costruisce i pulsanti e li aggiorna.
- **`CaricaOrarioEsempio()`:** crea 34 tessere, distribuite 7+7+7+7+6, senza buchi. I dati sono fittizi.
- **`CreaScacchiera()`:** crea i controlli grafici una sola volta.
- **`AggiornaScacchiera()`:** legge il modello e mostra materia e aula; non modifica l'orario.
- **`StessaPosizione()`:** confronta giorno e ora; con la classe `Posizione` tradizionale non usiamo `==` per confrontare il contenuto di due oggetti.
- **`Cella_Click()`:** gestisce solo la selezione.

- `AggiornaSelezione()`: colora i pulsanti e abilita/disabilita Scambia.
- `Scambia_Click()`: delega la decisione al motore e aggiorna la GUI.

7. Le classi del primo e secondo passo

Per compilare il progetto occorre includere **senza modifiche** i cinque file delle dispense precedenti. Le firme dei metodi usati dalla finestra sono:

```
// Orario
public Tessera Leggi(Posizione posizione);
public void Scrivi(Posizione posizione, Tessera tessera);

// MotoreScambi
public bool Scambia(Orario orario, Posizione prima, Posizione seconda);
```

Le righe qui sopra sono **solo un promemoria delle firme**, non codice da incollare in un file. Il motore del secondo passo esegue lo scambio provvisorio, chiama `ValidatoreOrario.SenzaBuchi()` e, se necessario, ripristina le tessere.

8. App.xaml e avvio

Visual Studio crea normalmente `App.xaml` e `App.xaml.cs`. Verificare che `App.xaml` avvii `MainWindow.xaml`:

```
<Application x:Class="OrarioClasse.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
    <Application.Resources>
    </Application.Resources>
</Application>
```

Il file `App.xaml.cs` generato da Visual Studio non richiede modifiche. Per compilare ed eseguire, premere **F5**.

9. Prove di funzionamento

L'orario iniziale contiene 7 lezioni dal lunedì al giovedì e 6 il venerdì. In ogni giorno le celle libere sono in fondo.

Prova	Azione	Risultato atteso
Selezione	Clic su una cella	Il pulsante cambia colore; Scambia resta disabilitato
Seconda selezione	Clic su una seconda cella	Entrambe evidenziate; Scambia abilitato
Deselezione	Nuovo clic su una cella selezionata	La selezione viene rimossa

Prova	Azione	Risultato atteso
Scambio valido	Scambia due lezioni occupate	Autorizzato; materia e aula cambiano posto
Spostamento valido	Scambia lunedì 7ª ora con venerdì 7ª ora vuota	Autorizzato: lunedì termina alla 6ª e venerdì alla 7ª
Spostamento non valido	Scambia lunedì 3ª ora con venerdì 7ª ora vuota	Rifiutato: lunedì avrebbe un buco alla 3ª
Due celle vuote	Scambia due celle vuote	Autorizzato, senza modifiche visibili

Attenzione: il motore del secondo passo non vieta lo scambio di due celle vuote né quello tra due posizioni identiche. La GUI impedisce di selezionare due volte la stessa cella, ma il motore non contiene ancora questo controllo. In futuro potremo decidere se tali richieste debbano essere rifiutate.

10. Concetti nuovi e conoscenze richieste

Concetto	Significato nel progetto
WPF	Libreria per realizzare interfacce grafiche Windows
XAML	Linguaggio basato su XML per descrivere la finestra
Code-behind	Codice C# associato alla finestra XAML
`Grid`	Contenitore che dispone controlli in righe e colonne
`Button`	Controllo selezionabile con un clic
`TextBlock` e `StackPanel`	Testo e disposizione verticale dei due dati della tessera
`Click`	Evento generato dal clic dell'utente
`sender`	Oggetto che ha generato l'evento
`Tag`	Proprietà usata per associare la posizione al pulsante
`Background`	Colore usato per evidenziare la selezione
`IsEnabled`	Abilita o disabilita il pulsante Scambia

11. Limiti intenzionali e sviluppi successivi

Questa versione non salva i dati su file, non gestisce più classi e non verifica vincoli su docenti o disponibilità delle aule. Il controllo implementato riguarda **solo l'assenza di buchi**. Inoltre il metodo `Orario.Scrivi()` è ancora pubblico: in un'evoluzione del progetto sarà opportuno impedire modifiche che aggirino il motore.

La finestra usa il **code-behind**, non MVVM: è una scelta didattica per introdurre gradualmente gli eventi WPF. Il modello e il motore restano comunque indipendenti dalla GUI e possono essere testati anche da console.

Risultato del terzo passo: una prima applicazione Windows funzionante, con 40 pulsanti, 34 tessere, 6 celle vuote, selezione di due celle, richiesta di scambio e aggiornamento della visualizzazione soltanto dopo la decisione del motore.